Qwen

A Protocol for Proving Character Movement in Godot: Synthesizing Inputs and Advancing Physics Frames in Headless Mode

This report provides a comprehensive analysis of the technical requirements for creating a deterministic validation gate for MILESTONE_006. The core objective is to establish a reliable method for proving that a single, synthetically triggered WASD-mapped input action produces a measurable position change on a `CharacterBody3D` node within a minimal, fixed number of physics frames. This research focuses exclusively on the two primary challenges identified: simulating input actions in a headless environment devoid of physical devices and ensuring sufficient physics processing occurs to register a meaningful state change. The scope adheres strictly to validating the fundamental link between an input action and a resulting kinematic displacement, excluding secondary concerns such as gameplay realism, complex movement behaviors, or edge-case handling for this initial milestone. The findings presented herein are based on an analysis of official documentation, community discussions, and source code patterns relevant to the Godot Engine.

Synthesizing Input Actions for Automated Testing

The central challenge in automating the validation of player movement lies in the simulation of user input within an environment where no physical hardware exists, such as in a headless CI/CD pipeline [27]. Standard input-handling functions like `Input.is_action_pressed()` rely on events generated by external devices like keyboards and mice [29]. When Godot is run in `--headless` mode, this event propagation system is inactive, rendering traditional input polling ineffective for automated tests [28] [29]. Consequently, any attempt to validate movement logic based on real-time input detection will fail silently, leading to a "reports success, proves nothing" scenario where the test passes but reveals no information about the actual functionality of the movement script . To overcome this limitation, it is essential to use an API

designed specifically for programmatic input generation, thereby decoupling the test from physical hardware and allowing for precise, repeatable control over the input stimuli sent to the game engine.

The Godot Engine provides a dedicated and robust API for this exact purpose: the `Input` singleton's `action_press()` and `action_release()` methods [4] [6]. These functions are not workarounds but are the officially sanctioned mechanism for synthetically triggering input actions from within a script [6]. By calling `Input.action_press("my_action")`, a developer can programmatically generate the same signal that would normally be produced by a user pressing a key mapped to "my_action" in the project's Input Map settings [59]. Similarly, `Input.action_release("my_action")` sends the corresponding release signal [4]. This capability is fundamental to any form of automated testing, AI-driven gameplay, or procedural event generation where direct user interaction is absent or undesirable. For the purposes of MILESTONE_006, this API provides the critical missing link, allowing the test script to definitively initiate the movement sequence by simulating the press of a 'move_forward' key.

The suitability of this approach is further validated when contrasted with alternative, less viable methods. Attempting to manually parse an input event using `Input.parse_input_event()` fails in headless mode because the event queue that this function reads from is not populated without a running display server [29]. Likewise, relying on `_input(event)` or `_unhandled_input(event)` callbacks is inappropriate for this use case. These functions are designed to react to incoming events, not to generate them proactively [32] [40]. Using them would require a more complex event-sending architecture that is far beyond the scope of a simple validation gate. The `action_press/release` pair offers a direct, synchronous, and highly predictable way to inject an action into the engine's input system. This ensures that the subsequent execution path within the `CharacterBody3D`'s script is driven entirely by the test's own logic, making the outcome deterministic and verifiable. The confirmation from multiple community sources underscores this is the standard practice for such tasks, with developers using these functions for simulated inputs in various contexts [4] [6].

A critical aspect of implementing this solution correctly involves understanding the nuances of Godot's input polling and timing mechanisms. While `action_press()` initiates the action, the choice of which function to check for that action's state within the `_physics_process` loop can have significant consequences. Functions like `is_action_just_pressed()` are designed to detect the very first frame a key is held down, but their behavior can be erratic due to timing mismatches between the rendering

and physics loops [15] [17]. If called inside `_physics_process`, it may miss inputs, especially at lower framerates, because its polling might not align perfectly with the frame the action was initiated [13] [15]. For example, if an action is pressed just before a physics frame starts, but the input poll happens at the very end of that frame, the action might appear to be pressed throughout the entire frame instead of being "just pressed" in a single one [35]. This can lead to inconsistent behavior where the action is registered in some frames but not others, even if the key is physically held down [34] [35].

To avoid these pitfalls and maintain the minimalist philosophy of the research goal, the most robust strategy is to perform the input simulation as a discrete, upfront action *before* the physics loop begins its execution. The test script will call `Input.action_press("move_forward")` once, and then immediately begin the physics advancement loop. Inside the `_physics_process` function of the character's script, the appropriate check should be `is_action_pressed()`, which returns true for as long as the action is considered pressed [66]. This continuous-state polling is unaffected by the subtle timing discrepancies that plague `is_action_just_pressed()` [66]. This approach simplifies the test logic immensely: there is one clear moment of input initiation, followed by a period of physics simulation during which the effect of that single, sustained press is observed. This directly addresses the research goal of proving that "a WASD-mapped input action... produces a real position change," without introducing the complexity and potential unreliability of trying to catch a transient "just-pressed" moment. The combination of a pre-scripted `action_press` and a continuous `is_action_pressed` check creates a clean, unambiguous, and highly reliable feedback loop perfect for an automated validation gate.

Method	Mechanism	Suitability for Headless Validation	Key Advantage	Key Disadvantage
`Input.action_press()` / `action_release()`	Programmatic injection of input actions into the engine's internal state [4] [6].	**High**: This is the intended and official method for synthetic input.	Direct, deterministic, and part of the core engine API.	Requires manual management of press/release states.
`Input.parse_input_event()`	Manual parsing of a generated `InputEvent` object [29].	**None**: Fails in `--headless` mode as the event queue is not active.	Not applicable.	Inactive in headless environments.
`_input()` / `_unhandled_input()` Callbacks	Event-driven functions that react to incoming events [40].	**Low**: Designed for reacting to real events, not generating them.	Can handle custom events.	Unsuitable for proactive, scripted input simulation.
`is_action_just_pressed()` Check	Polls for the first frame an action becomes active [15] [17].	**Variable/Poor**: Prone to timing issues and missed inputs, especially in `_physics_process` [13].	Good for detecting discrete button presses in UI/menu contexts [66].	Unreliable for consistent movement checks; sensitive to timing [15].
`is_action_pressed()` Check	Polls for the current state of an action (whether it is currently held) [66].	**High**: Provides a stable, continuous state that is not affected by timing glitches.	Robust and reliable for checking sustained inputs like holding down a movement key.	Does not detect the initial press event, only the ongoing state.

This comparative analysis solidifies the conclusion that `Input.action_press()` is the unequivocally correct tool for this task. Its design purpose aligns perfectly with the need for deterministic, headless testing. By using it to trigger the action before the physics simulation begins, and pairing it with `is_action_pressed()` within the `_physics_process` for state checking, the validation gate can reliably prove the connection between a user-defined action and the character's response, forming the bedrock of MILESTONE_006.

Advancing Physics Frames with Deterministic Control

Ensuring that the physics simulation has progressed sufficiently after an input is provided is the second pillar of a valid movement test. A naive implementation that exits immediately after simulating the input would yield no useful information, as the `CharacterBody3D`'s velocity would not have had time to integrate into a new position [2]. The movement logic, typically residing in the `_physics_process(delta)` function, requires multiple executions to produce a measurable result [16]. Therefore, the

validation gate must contain a mechanism to hold the headless Godot process open for a predetermined, minimal number of physics steps. The user's directive to use a hardcoded frame count rather than a configurable or time-based duration is crucial for maintaining the simplicity and definitive pass/fail nature of the test. This approach transforms the validation from a vague "seems to work" assessment into a concrete, reproducible experiment.

Godot's architecture separates the rendering loop from the physics loop, a concept central to achieving framerate-independent simulations [14] [65]. The physics loop runs at a fixed interval defined in the project settings under `Project Settings > Physics > Common > Physics FPS`, which defaults to 60 times per second [42] [51]. This means that each `_physics_process(delta)` call represents a discrete, consistent timestep of the physics world, independent of how many frames the GPU renders per second [63]. This separation is precisely what enables the desired level of control. Instead of waiting for a variable amount of wall-clock time, the test can synchronize itself directly with the engine's physics ticker.

The recommended method to advance the simulation by a fixed number of physics frames is to implement a simple loop that polls the tree's `physics_frame` property . Each iteration of the loop waits for the next physics frame to begin. The `await get_tree().physics_frame` statement is a coroutine that yields control until the next physics step is initiated by the engine . By placing this statement inside a loop that iterates a hardcoded number of times (e.g., `for i in range(5): await get_tree().physics_frame`), the script can pause execution for exactly five physics timesteps. This technique directly satisfies the research goal's requirement for a non-configurable, fixed frame count. It guarantees that the character's movement script will have executed its logic at least five times, providing ample opportunity for acceleration, velocity integration, and collision checks to occur. This deterministic waiting mechanism is superior to alternatives like `yield(get_tree(), 'process_frame')`, which is tied to the rendering loop and thus subject to framerate fluctuations, or using timers, which introduces unnecessary complexity and reliance on delta time calculations that defeat the purpose of a simple, fixed-step test [18] [43].

The choice of the minimal frame count (N) is a pragmatic consideration. While theoretically, a single physics frame might be enough for a simple velocity change to register, practical experience suggests that newly instantiated physics bodies can exhibit transient behaviors [70]. Some developers even implement logic to skip the first few frames of a character's life to allow its state to stabilize [2]. There can also be a slight delay between when a physics object is created and when it is properly synchronized with

the physics server, potentially resulting in a default or incorrect position on its first frame [1]. Therefore, setting a small, hardcoded value like N=2, N=3, or N=5 provides a safety margin. This ensures that the test does not fail due to a race condition related to initialization and gives the movement logic a high probability of having executed fully before the final position assertion is made. The value of N is not critical to gameplay realism but serves as a buffer to account for these minor engine-level delays, making the test more robust. For instance, N=5 provides a conservative buffer, while N=2 might be sufficient for well-behaved scripts, balancing speed and reliability.

This approach of hardcoding a physics frame count is fundamentally different from attempting to time the movement. Time-based approaches (`yield(get_tree(), 'process_frame')`) are tied to the rendering loop, meaning a test could pass on a machine running at 200fps but fail on another running at 30fps simply because the physics simulation had fewer opportunities to run [12]. The physics frame-based approach eliminates this variability. Whether the game is running at 60, 120, or 200 frames per second, five calls to `await get_tree().physics_frame` will always correspond to the same amount of simulated physics time ($5/\text{PhysicsFPS}$ seconds). This consistency is paramount for a validation gate whose sole purpose is to provide a binary pass/fail result based on a deterministic set of conditions. It removes environmental variables and focuses purely on the correctness of the code under test. The combination of a synthesized input and a deterministically advanced physics simulation creates a controlled laboratory environment within Godot, allowing for the isolation and verification of the specific behavior required by MILESTONE_006.

Aspect	Recommended Approach (`await get_tree().physics_frame`)	Alternative Approaches	Rationale for Choice
Mechanism	Polls a built-in engine property to wait for the next physics tick .	`yield(get_tree(), 'idle_frame')`, timers, or manual frame counters.	Directly synchronizes with the physics loop, not the rendering loop [63].
Determinism	**High**: Guarantees a fixed number of physics timesteps regardless of render framerate [51].	**Low**: Highly dependent on render framerate, leading to inconsistent results across machines [12].	Essential for a reliable, reproducible validation gate with a clear pass/fail outcome.
Configuration	**None**: Uses a hardcoded integer in a loop (e.g., `for i in range(5): ...`).	**Required**: Often requires setting a target duration or framerate, adding complexity.	Aligns with the "prove the minimum" philosophy; avoids scope creep associated with configurability .
Implementation Complexity	**Simple**: A single line of code inside a standard loop construct.	**Complex**: Requires timer logic, delta time management, or custom frame tracking systems.	Keeps the test script lightweight and focused on the core validation logic.
Robustness	**High**: Accounts for engine initialization delays by allowing for a small, safe buffer of frames [1] [2].	**Variable**: Time-based waits may be too short if the physics loop is slow or delayed.	Ensures the character's script has had sufficient time to execute its movement logic completely.

By adopting the `await get_tree().physics_frame` pattern, the validation gate achieves the dual goals of precision and simplicity. It provides absolute control over the simulation timeline, enabling a definitive assertion about the character's state after a known quantity of physics processing. This makes it the ideal solution for the foundational movement test required by MILESTONE_006.

A Complete Protocol for Movement Validation

Synthesizing the solutions for input simulation and physics frame advancement allows for the formulation of a complete, three-step protocol for the MILESTONE_006 validation gate. This protocol is designed to be deterministic, self-contained, and directly address the research goal: proving that a single synthetically triggered action results in a measurable position change. It isolates the variable of interest (the input-action-to-position-change link) and executes a controlled experiment with a clear pass/fail outcome. The entire process unfolds within a single headless test script, making it ideal for integration into an automated build pipeline.

Step 1: Scene Setup and Initial State Capture The first step involves preparing the test environment. This begins by instantiating the scene containing the `CharacterBody3D` node that will be tested. This scene should be constructed via the text-templating approach described in the project's architecture, ensuring its structure is known and reproducible . Once the scene is added to the main scene tree, the test script must immediately capture the initial state of the character. The most critical piece of data to record is the `global_position` or `position` of the `CharacterBody3D`. This initial vector serves as the baseline against which all subsequent changes will be measured. Storing this value in a variable (e.g., `initial_position`) is essential for the final assertion. This step ensures that any change observed later is not due to an initial offset or improper instantiation.

Step 2: Input Simulation and Physics Execution With the initial state captured, the protocol proceeds to the core of the experiment. This step is divided into two distinct phases: input simulation and physics execution. First, the script programmatically triggers the desired movement action using the `Input` API. Specifically, it calls `Input.action_press("move_forward")`, where "move_forward" is the name of the action mapped to the W key in the project's Input Map [58] [59]. This single line of code generates the synthetic input event that would normally be produced by a player pressing the 'W' key. Immediately following this, the script must execute the physics simulation for

a fixed number of frames. As established previously, this is accomplished by entering a loop that awaits the tree's `physics_frame` property N times, where N is a small, hardcoded integer (e.g., N=5) . During each iteration of this loop, the Godot engine will execute the `_physics_process(delta)` function of the `CharacterBody3D`. Within this function, the character's movement script will run, likely integrating a velocity vector based on the `is_action_pressed("move_forward")` check. After N iterations, the physics simulation will have advanced by N timesteps, and the character will have had a chance to move if the input was processed correctly.

Step 3: Position Assertion and Test Outcome The final step is to verify whether the simulation produced the expected result. After the physics loop completes, the test script must read the `CharacterBody3D`'s new position. This current position should be stored in a variable (e.g., `final_position`). The core of the assertion is a comparison between `final_position` and the `initial_position` captured in Step 1. Since floating-point arithmetic can introduce tiny imprecisions, a direct equality check (`==`) is unreliable. Instead, the test should calculate the difference along the axis of movement (e.g., `abs(final_position.z - initial_position.z)` for forward/backward movement) and compare this delta to a small, predefined threshold (epsilon). If the calculated delta is greater than this epsilon value, the test passes, confirming that a measurable position change occurred as a result of the input action. Conversely, if the delta is less than or equal to the epsilon, the test fails, indicating that the input action did not produce the expected kinematic response. This binary outcome provides a clear, unambiguous verdict on the functionality of the movement code, fulfilling the primary objective of the validation gate. This entire sequence—setup, simulate, execute, assert—is encapsulated within a single, autonomous test script that can be run in `--headless` mode, providing a powerful and repeatable quality assurance check.

This protocol elegantly enforces the strict "prove the minimum" discipline outlined in the research goal. It focuses solely on the question of whether a single action maps to a single displacement. It deliberately avoids complexities like input timing variations, chaining multiple actions, or handling slopes and collisions, as these are separate problems that warrant their own milestones and tests. By keeping the test atomic and focused, it prevents the common pitfall of building a test that grows too broad and eventually becomes a vague "movement seems fine" check instead of a rigorous proof of a specific, isolated behavior. The protocol is a blueprint for a unit test for the character's movement logic, providing a gold-standard method for validating MILESTONE_006.

Secondary Considerations: .tscn Serialization and Broader Applications

While the primary focus of this research is the immediate validation of character movement, a thorough analysis of the supporting materials reveals several secondary considerations with significant implications for the project's overall architecture and future development. These insights, though not part of MILESTONE_006 itself, provide valuable context and highlight best practices that can prevent future bugs and streamline the creation of additional validation gates. Two key areas emerge: the importance of authoritative `.tscn` file serialization rules and the broader applicability of the synthesized-input-and-physics-waiting pattern to other game mechanics.

The first consideration revolves around the `.tscn` file format, which is used to serialize Godot scenes and resources into a human-readable text format. The user's prior experience with a bug caused by hand-templating a `Transform3D` matrix highlights a critical vulnerability in this process . The documentation notes that the `.tscn` format is not formally specified in a public document, leading to reliance on secondhand knowledge from tutorials and forum posts, which can be inaccurate . This lack of an official specification makes manual templating a fertile ground for subtle, hard-to-diagnose errors. For instance, the `Transform3D` bug was likely caused by an incorrect ordering of basis vectors or translation components when writing the text representation by hand .

To mitigate this risk, the most reliable strategy is to adopt a "ground truth" approach. Instead of guessing the correct serialization format, the project should leverage the Godot editor itself as the ultimate authority on the format. The recommended workflow is to create a known-good template of a new node type (e.g., a new piece type for the game) directly within the Godot editor, saving it as a `.tscn` file . This generated file is the canonical reference. The Python builder script's output, which uses text templates, should then be compared against this ground-truth file. Any differences can be analyzed to refine the templating logic, ensuring that the generated files are identical to those produced by the editor. This practice effectively outsources the complex and ever-changing details of the `.tscn` serialization format to the engine's own C++ code, which is found in files like `scene/resources/resource_format_text.cpp` . This is the most authoritative source for the format's rules, as it is literally the code that writes the files . Adopting this diff-based validation for all new scene and resource templates provides a powerful defense against the silent failures that can arise from hand-templating bugs, improving the robustness of the entire build system.

The second, and perhaps more impactful, secondary consideration is the broader application of the validation protocol developed for MILESTONE_006. The techniques of synthetic input simulation and deterministic physics stepping are not unique to character movement; they constitute a reusable pattern for unit-testing a wide variety of game mechanics in a headless environment. This pattern can be applied to validate the behavior of other nodes and systems mentioned in the conversation history, transforming them from abstract concepts into verifiable, testable components. For example, to create a validation gate for an `exit_trigger` node (which might be an `Area3D`), the same protocol can be adapted:

1. **Setup:** Instantiate the `exit_trigger` `Area3D` node and a `CharacterBody3D` at a position outside of its collision shape.
2. **Simulation & Execution:** Programmatically move the `CharacterBody3D` directly into the `Area3D`'s bounds by setting its `global_position` property. Then, advance the simulation by a few hardcoded physics frames (e.g., N=3) using the `await get_tree().physics_frame` loop.
3. **Assertion:** Check if the `body_entered` signal from the `Area3D` was emitted. This can be done by having the test script connect to the signal beforehand and setting a boolean flag upon emission. If the flag is true after the physics loop, the test passes, confirming that the trigger correctly detects when a body enters its volume.

This demonstrates the power of the pattern: it provides a generic framework for testing cause-and-effect relationships within the game's logic. The "cause" is either a synthetic input (`action_press`) or a direct state change (moving a node), and the "effect" is an observable outcome, such as a position change or a signal emission. By codifying this pattern, the project can systematically build a suite of automated tests for all major game systems, moving towards a more robust and maintainable development process. This modular testing approach, where each component is verified in isolation before being integrated, is a cornerstone of modern software engineering and is directly applicable to game development.

These secondary considerations reinforce the soundness of the primary research conclusions. The chosen methods for input simulation and physics control are not only the correct solution for MILESTONE_006 but also represent a scalable and sustainable architectural pattern for the project's future. By simultaneously addressing the immediate needs of the milestone and laying the groundwork for future testing efforts, the project can ensure both short-term success and long-term stability.

Final Recommendations for Milestone Implementation

Based on the comprehensive analysis of the provided materials, a clear and actionable path emerges for implementing the MILESTONE_006 validation gate. The primary goal—to prove that a single, synthetically triggered WASD-mapped action produces a measurable position change on a `CharacterBody3D` within a minimal, fixed number of physics frames—is achievable through a straightforward and robust protocol. The recommendations below synthesize the key findings into a direct implementation plan, while also incorporating the secondary considerations that enhance the project's overall architectural integrity.

First and foremost, the implementation of the validation gate should proceed using the three-step protocol detailed previously. The test script must: 1. **Instantiate the test scene** containing the `CharacterBody3D` and immediately **capture its initial `position`**. 2. **Programmatically trigger the input** by calling `Input.action_press("move_forward")` (or the relevant action name). 3. **Execute the physics simulation** for a hardcoded, minimal number of frames (e.g., N=5) by using a loop with `for _ in range(N): await get_tree().physics_frame`. 4. **Assert the outcome** by comparing the character's final `position` to the initial one. The assertion should pass if the displacement along the movement axis exceeds a small epsilon value, and fail otherwise.

This implementation directly leverages the `Input.action_press()` and `get_tree().physics_frame` APIs, which are the correct and most reliable tools for this task in a headless environment. It adheres to the "prove the minimum" philosophy by focusing on a single, isolated cause-and-effect relationship, thereby avoiding the pitfalls of overly complex or ambiguous tests. The use of a hardcoded frame count ensures the test is deterministic and provides a clear, binary pass/fail result, which is essential for an automated validation gate.

Second, to prevent the recurrence of silent bugs related to resource serialization, the project's build system should adopt the "ground truth" methodology for generating `.tscn` files. For every new node or scene type added to the game, a known-good template should be created by hand in the official Godot editor. This editor-generated file should then be used as the canonical reference against which the output of the Python templating script is diffed. This practice effectively uses the engine itself as the ultimate authority on the `.tscn` format, bypassing the ambiguities of informal documentation and preventing subtle serialization errors that can be difficult to debug .

Finally, the principles established by this research should be treated as a template for future validation gates. The synthesized-input-and-physics-waiting pattern is a versatile and powerful technique for unit-testing various game mechanics. The team should proactively identify other systems—such as triggers, collectibles, or enemy behaviors—and apply this same protocol to create deterministic, automated tests for them. This will foster a culture of continuous verification, improve the overall quality of the codebase, and significantly reduce the risk of regressions as the project scales.

In summary, the successful implementation of MILESTONE_006 hinges on executing a simple yet precise validation experiment. By using the Godot engine's built-in capabilities for synthetic input and physics synchronization, and by adopting rigorous practices for asset templating and future testing, the project can achieve its immediate goal while also strengthening its foundation for long-term success.

Reference

1. Instantiated object's first physics frame has wrong position https://forum.godotengine.org/t/instantiated-objects-first-physics-frame-has-wrong-position/73953
2. CharacterBody3D in air for the first couple of frames? : r/godot - Reddit https://www.reddit.com/r/godot/comments/1hesexc/characterbody3d_in_air_for_the_first_couple_of/
3. CharacterBody3D: Movement :: Godot 4 Recipes - KidsCanCode.org https://kidscancode.org/godot_recipes/4.x/3d/characterbody3d_examples/index.html
4. How can I simulate inputs? : r/godot - Reddit https://www.reddit.com/r/godot/comments/s04tk0/how_can_i_simulate_inputs/
5. [4.3.beta3]: _physics_process() does not detect Input ... - GitHub https://github.com/godotengine/godot/issues/94409
6. How to simulate a button press in code? - Godot Forums https://godotforums.org/d/19196-how-to-simulate-a-button-press-in-code
7. Using the default CharacterBody3D movement script, how can you ... https://www.reddit.com/r/godot/comments/1b4fm9p/using_the_default_characterbody3d_movement_script/

8. How to implement deceleration with a CharacterBody3D node via ... https://forum.godotengine.org/t/how-to-implement-deceleration-with-a-characterbody3d-node-via-code/105618
9. 3D Movement in Godot 4 (Simple Guide) - YouTube https://www.youtube.com/watch?v=oP4KCPtbg3g
10. Godot 4 3D Platformer Lesson #3: Player Object + Movement/Jump ... https://www.youtube.com/watch?v=T_7ob1zAjLo
11. Player Movement and Camera Rotation not happening together in ... https://stackoverflow.com/questions/78029612/player-movement-and-camera-rotation-not-happening-together-in-godot-3d
12. _process(), _physics_process() and Input. : r/godot - Reddit https://www.reddit.com/r/godot/comments/1cmp6q1/process_physics_process_and_input/
13. `Input.is_action_just_pressed` dropping inputs at low framerates https://github.com/godotengine/godot/issues/73339
14. Understanding Process versus Physics Process | Godot 3.x - YouTube https://www.youtube.com/watch?v=UrbcDJFLPc0
15. Dropped inputs with .is_action_just_pressed() in _physics_process ... https://community.gamedev.tv/t/dropped-inputs-with-is-action-just-pressed-in-physics-process-in-old-godot/231462
16. Godot: _process() vs _physics_process(): I need a valuable example https://stackoverflow.com/questions/73098693/godot-process-vs-physics-process-i-need-a-valuable-example
17. Fix: Godot Input.is_action_just_pressed() Triggering Multiple Times https://bugnet.io/blog/fix-godot-input-action-just-pressed-multiple-times
18. Why I Ban Logic from _process and _physics_process (and Use ... https://medium.com/go-go-godot-game-dev/why-i-ban-logic-from-process-and-physics-process-and-use-state-machines-instead-in-godot-7c4ba958bd4b
19. Input Handling: _process vs _physics_process - Godot Forums https://godotforums.org/d/36935-input-handling-process-vs-physics-process
20. How to move a sprite in Godot 4 WASD and D-Pad Tutorial - YouTube https://www.youtube.com/watch?v=KQ5GDP6qdP0
21. Change movement to WASD - Programming - Godot Forum https://forum.godotengine.org/t/change-movement-to-wasd/56574
22. How to change arrow keys to WASD? : r/godot - Reddit https://www.reddit.com/r/godot/comments/u4m2g3/how_to_change_arrow_keys_to_wasd/
23. Godot Player Movement | Smooth WASD 2D Movement - YouTube https://www.youtube.com/watch?v=SUZpVd18IMM

24. Editor 3D WASD Navigation often stops working #73386 - GitHub https://github.com/godotengine/godot/issues/73386
25. Smooth WASD 2D Movement in 10 lines of code : r/godot - Reddit https://www.reddit.com/r/godot/comments/fue9tv/godot_player_movement_smooth_wasd_2d_movement_in/
26. How to make movement relative to the looking direction? https://godotforums.org/d/32770-how-to-make-movement-relative-to-the-looking-direction
27. Is there a Godot 4 headless for CI/CD - Reddit https://www.reddit.com/r/godot/comments/yojy1b/is_there_a_godot_4_headless_for_cicd/
28. Command line tutorial - Godot Docs https://docs.godotengine.org/en/latest/tutorials/editor/command_line_tutorial.html
29. `Input.parse_input_event(event)` do not work on headless mode https://github.com/godotengine/godot/issues/73557
30. CI-tested GUT for Godot 4: fast, green, and reliable - Medium https://medium.com/@kpicaza/ci-tested-gut-for-godot-4-fast-green-and-reliable-c56f16cde73d
31. Using CharacterBody2D/3D - Godot Docs https://docs.godotengine.org/en/4.4/tutorials/physics/using_character_body_2d.html
32. Bug with is_action_just_pressed or _unhandled_input · Issue #95162 https://github.com/godotengine/godot/issues/95162
33. Persistent Input key pressed Bug/Glitch · Issue #19137 - GitHub https://github.com/godotengine/godot/issues/19137
34. is_action_pressed sometimes skips key press · Issue #36396 - GitHub https://github.com/godotengine/godot/issues/36396
35. [3.x] _input reports is_action_just_pressed() multiple times per frame ... https://github.com/godotengine/godot/issues/97526
36. Input.is_action_pressed(leftMouse) only works when moving mouse ... https://www.reddit.com/r/godot/comments/12dc8bq/inputis_action_pressedleftmouse_only_works_when/
37. A bunch of helpful tips from my first 2 weeks using Godot 4 - Reddit https://www.reddit.com/r/godot/comments/spu6sv/a_bunch_of_helpful_tips_from_my_first_2_weeks/
38. Separate input polling from render thread · Issue #1288 - GitHub https://github.com/godotengine/godot-proposals/issues/1288
39. func _input vs Input.is_action_pressed inside func _physics_process ... https://www.reddit.com/r/godot/comments/fg8rd5/func_input_vs_inputis_action_pressed_inside_func/

40. Use _Input(event) or _physics_process(delta) - Performance? https://forum.godotengine.org/t/use-input-event-or-physics-process-delta-performance-input-lag-isnt-a-problem/13354
41. func _process(delta): velocity = Input.get_vector("ui_left", "ui_right ... https://www.facebook.com/groups/godotengine/posts/2993318374138070/
42. _process() and _physics_process() in Godot - Kehom's Forge http://kehomsforge.com/tutorials/single/process-physics-process-godot/
43. When and when NOT to use delta in _process and ... - GameDev.tv https://community.gamedev.tv/t/when-and-when-not-to-use-delta-in-process-and-physics-process-functions/231393
44. I don't understand what is wrong (Update, I do) - Godot Forums https://godotforums.org/d/35520-i-dont-understand-what-is-wrong-update-i-do
45. Understanding 'delta' :: Godot 4 Recipes - KidsCanCode.org https://kidscancode.org/godot_recipes/4.x/basics/understanding_delta/index.html
46. Moving the player with code - Godot Docs https://docs.godotengine.org/en/stable/getting_started/first_3d_game/03.player_movement_code.html
47. Tutorial: First Person Movement In Godot 4 - YouTube https://www.youtube.com/watch?v=v4IEPi1c0eE
48. Character Movement in 3D | GDQuest Library https://www.gdquest.com/library/character_movement_3d_platformer/
49. Godot 4: 3D Character Controller - YouTube https://www.youtube.com/watch?v=AW3rT-7J8ag
50. 命令行教程— Godot Engine (4.3) 简体中文文档 https://docs.godotengine.org/zh-cn/4.3/tutorials/editor/command_line_tutorial.html
51. Idle and Physics Processing - Godot Docs https://docs.godotengine.org/en/stable/tutorials/scripting/idle_and_physics_processing.html
52. Input::is_action_just_pressed has problematic behaviour #58465 https://github.com/godotengine/godot/issues/58465
53. Is polling input frame independent? - Archive - Godot Forum https://forum.godotengine.org/t/is-polling-input-frame-independent/24478
54. True Framerate Independent Input : r/godot - Reddit https://www.reddit.com/r/godot/comments/1kxz4n0/true_framerate_independent_input/
55. Setting RigidBody global_transform into the `_input` function fails. https://github.com/godotengine/godot/issues/40486
56. Input Handling in Godot is Surprisingly Complex! So I did a DEEP ... https://www.youtube.com/watch?v=AfX00eQIRCU

57. Using InputEvent — Godot Engine (stable) documentation in English https://docs.godotengine.org/en/stable/tutorials/inputs/inputevent.html
58. Player scene and input actions - Godot Docs https://docs.godotengine.org/en/stable/getting_started/first_3d_game/02.player_input.html
59. InputMap — Godot Engine (stable) documentation in English https://docs.godotengine.org/en/stable/classes/class_inputmap.html
60. What's the difference between _process and _physics_process in ... https://www.facebook.com/groups/godotengine/posts/1394758933994030/
61. _process vs _physics_process | Godot Nuggets - YouTube https://www.youtube.com/watch?v=murrsm0rB1Q
62. Using CharacterBody2D/3D - Godot Docs https://docs.godotengine.org/en/stable/tutorials/physics/using_character_body_2d.html
63. Node — Godot Engine (stable) documentation in English https://docs.godotengine.org/en/stable/classes/class_node.html
64. Overly nitpicky questions about input processing - Help - Godot Forum https://forum.godotengine.org/t/overly-nitpicky-questions-about-input-processing/123805
65. Introduction - Xogot | Documentation https://docs.xogot.com/documentation/xogot/physics_interpolation_introduction/
66. Fix: Godot Input is_action_just_pressed Misses After Game Pause https://bugnet.io/blog/fix-godot-input-action-just-pressed-misses-after-game-pause
67. Using CharacterBody2D/3D - Godot Docs https://docs.godotengine.org/ja/4.x/tutorials/physics/using_character_body_2d.html
68. Using CharacterBody2D/3D - Godot Docs https://docs.godotengine.org/es/4.x/tutorials/physics/using_character_body_2d.html
69. Character Jitters after reaching position - Navigation - Godot Forum https://forum.godotengine.org/t/character-jitters-after-reaching-position/124235
70. Anyone else having problems with CharacterBody3D in Godot 4? https://www.reddit.com/r/godot/comments/yq3yka/anyone_else_having_problems_with_characterbody3d/
71. Moving without pressing keys using modified godot 3d basic script https://stackoverflow.com/questions/79620621/moving-without-pressing-keys-using-modified-godot-3d-basic-script
72. Godot NavigationAgent2D Setup & Common Bugs - Vav Labs https://vav-labs.com/blog/navigationagent-setup-and-common-gotchas-in-godot/